

链表

(2022) 已知不带头结点的双向链表 head，请在不借助辅助空间的情况下，设计一个倒置该双向链表的算法。其中，双向链表的结点类型定义如下：

```
typedef struct DLNode {  
    int data;  
    DLNode *prev;  
    DLNode *next;  
}DLNode;
```

```
void reverseList(DLNode *&head) {  
    DLNode *p = head, *pr = NULL;  
    if(p == NULL || p->next == NULL) return;  
    while(p) {  
        head = p->next;  
        p->next = p->prev;  
        p->prev = head;  
        pr = p;  
        p = head;  
    }  
    head = pr;  
}
```

(2021) 删除带头结点的单循环链表中值等于x结点，成功返回1，否则返回0

```
int Deletex(LNode *&head, int x) {  
    LNode *p = head->next; // p指向第一个结点  
    LNode *pre = head;     // pre指向p的前驱结点  
    while (p != head) {    // 遍历整个循环链表  
        if (p->data == x) { // 找到了值等于x的结点  
            pre->next = p->next; // 删除该结点  
            delete p;           // 释放被删除结点的内存  
            return 1;           // 返回1表示删除成功  
        }  
        pre = p;  
        p = p->next; // 继续遍历下一个结点  
    }  
    return 0; // 没有找到值等于x的结点，返回0表示删除失败  
}
```

(2020) 单循环链表去重，返回具有相同值元素个数

```
//自己写的，不保证正确  
int DeleteNode(LNode *&head) {  
    int count = 0;  
    LNode *cur = head->next;  
    while (cur != head) {  
        LNode *pr = cur;  
        LNode *p = cur->next  
        bool flag=false;  
        while (p != head) {
```

```

        if(p->data==cur->data){
            flag=true;
            pr->next = p->next;
            delete p;
            p = pr->next;

        }else{
            pre = p;
            p = p->next;
        }
    }
    if(flag)count++;
    cur=cur->next;
}
return count;
}

```

(2018) 合并2个带头结点的单循环列表ha和hb, 按指递增排序

```

void intersection(lnode *&ha, lnode *&hb) {
    lnode *pa = ha->next, *pr = ha, *pb = hb->next, *s;
    while(pa != ha && pb != hb) {
        if(pa->data < pb->data) {
            pr = pa;
            pa = pa->next;
        } else if(pa->data == pb->data) {
            pr = pa;
            pa = pa->next;
            pb = pb->next;
        } else {
            s = new lnode;
            s->data = pb->data;
            pr->next = s;
            s->next = pa;
            pr = s;
            pb = pb->next;
        }
    }
    while(pb != hb) {
        s = new lnode;
        s->data = pb->data;
        s->next = pr->next;
        pr->next = s;
        pr = s;
        pb = pb->next;
    }
}

```

(2016) 已知一个带头结点的按值非递减的单循环链表, 结点类型为 (data,next) 。以 head 为头指针, 每个结点的 data 域存放的是一个整数,试构造一个删除所有值大于min, 小于max的结点的算法。函数原型为void rangedelete (lnode *&head,int min,int max)。

```

void rangedelete(lnode *&head, int min, int max) {
    lnode *pr = head, *p = head->next;
    while(p != head && p->data <= min) {
        pr = p;
        p = p->next;
    }
    while(p != head && p->data < max) {
        pr->next = p->next;
        delete p;
        p = pr->next;
    }
}

```

(2016) 已知一个带头结点的单链表ha中存放一组整型数，结点类型为(data,next)，构造一算法，将链表ha中值为偶数的结点加入链表hb中，且链表hb按值非递减排列。函数原型为void inserthb(lnode *ha, lnode *&hb)。

```

void inserthb(lnode *ha, lnode *&hb) {
    hb = new lnode;
    hb->next = NULL;
    lnode *p = ha->next, *qr, *q, *s;
    while(p) {
        if(p->data % 2 == 0) {
            qr = hb;
            q = hb->next;
            while(q && q->data <= p->data) {
                qr = q;
                q = q->next;
            }
            s = new lnode;
            s->data = p->data;
            s->next = q;
            qr->next = s;
        }
        p = p->next;
    }
}

```

(2015) 有带头结点的递增有序单链表heada和headb，存储的分别为数据集A和B，要求完成求差集A-B的算法void difference (heada, headb, n)（即完成从A的链表中删除B链表中数据的工作），同时返回最后A集合的元素个数n。

```

void difference(heada, headb, n) {
    //n是结果集合中元素的个数，调用时为0
    p = heada->next;    //链表A的工作指针
    q = headb->next;    //链表B的工作指针
    pre = heada;        //pre为A中p所指结点的前驱结点的指针
    while (p != NULL && q != NULL) {
        if (p->data < q->data) { //A链表中当前结点指针后移
            pre = p;
            p = p->next;
            n++;
        } else if (p->data > q->data) //B链表中当前结点指针后移
            q = q->next;
    }
}

```

```

        else { //处理A, B中元素值相同的结点, 应删除
            pre->next = p->next;
            u = p;
            p = p->next;
            free(u);
        }
    }
}

```

二叉树

(2022) 二叉树的存储结构如下:

```

typedef struct BitNode{
    int data;
    BitNode *lchild;
    BitNode *rchild;
}BitNode;

```

试用类栈 Stack 中方法 EmptyStack() (栈的判空函数, 若空返回 1, 否则返回 0), Push(e)(压栈), Pop(e) (退栈) 构造计算二叉树中度为 2 结点数的非递归算法。函数原型为 int CountNode(BitNode *t)。

```

int CountNode(BitNode *t) {
    Stack s;
    BitNode *p = t;
    int count = 0;
    while(p || !s.EmptyStack()) {
        if(p) {
            if(p->lchild && p->rchild) count++;
            s.Push(p);
            p = p->lchild;
        } else {
            s.Pop(p);
            p = p->rchild;
        }
    }
    return count;
}

```

(2021) 计算二叉树树高的递归算法

```

int hightbitree(BitNode* t) {
    if (t == NULL) return 0;
    return max(hightbitree(t->lchild),
               hightbitree(t->rchild)) + 1;
}

```

(2020) 判断两颗二叉树是否相似的递归算法, 相似返回1, 否则返回0

```
int IsAlike(bitnode* t, bitnode* s) {
    if (t == NULL && s == NULL) return 1;
    else if (t == NULL || s == NULL) return 0;
    else return IsAlike(t->left, s->left) &&
        IsAlike(t->right, s->right);
}
```

(2018) 返回二叉排序中最小结点地址的递归和非递归算法

```
//递归
BITNode *FindMin(BITNode *t) {
    if(t == NULL) return t;
    else if(t->lchild == NULL) return t;
    else return FindMin(t->lchild);
}
//非递归
BITNode *FindMin(BITNode *t) {
    if(t != NULL)
        while(t->lchild != NULL)
            t = t->lchild;
    return t;
}
```

(2016) 二叉树采用二叉链表作为存储结构，链表中结点的结构为：

```
typedef struct bitNode{
    int data;
    bitnode *lt;
    bitnodd *rt;
}bitNode;
```

构造一个计算二叉树中度为2且结点的值小于x的结点个数的递归算法。函数原型为int countx(bitNode *t, int x)。

```
int countx(bitnode *t, int x) {
    if(t) {
        if(t->lt && t->rt && t->data < x)
            return 1 + countx(t->lt, x) + countx(t->rt);
        else return countx(t->lt, x) + countx(t->rt);
    } else return 0;
}
```

(2016) 二叉树的存储结构如下：

```
typedef struct BITNode{
    int data;
    BITNode *lt;
    BITNode *rt;
}BITNode;
```

试构造一个拷贝二叉树的递归算法。函数原型为void btcopy(BITNode *s,BITNode *&t)。(注s为原二叉树，t为拷贝后的二叉树)。

```
void btcopy(BITNode *s, BITNode *&t) {  
    if(s == NULL)t = NULL;  
    else {  
        t = new BITNode;  
        t->data = s->data;  
        btcopy(s->lt, t->lt);  
        btcopy(s->rt, t->rt);  
    }  
}
```